Project Setup & Feature Development Guide

AI-Based Design Change Impact Analysis System

1. Introduction

This document explains:

- How a new developer can set up the project
- How to add new features in the system

The system has 4 main parts:

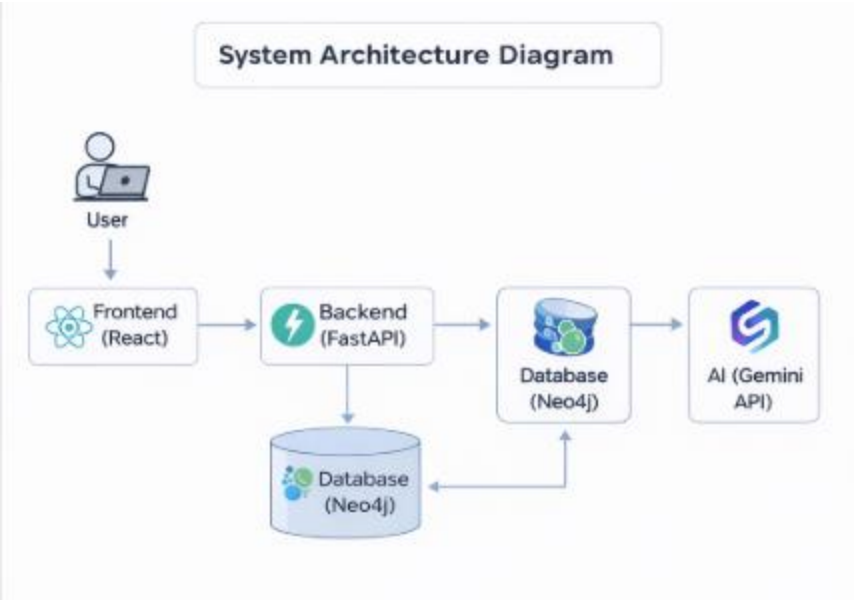
- Frontend (React)
- Backend (FastAPI)
- Database (Neo4j)
- AI (Gemini API)

2. System Overview

The system takes a design change as input and gives:

- Which systems are affected
- Cost estimation (P10, P50, P90)
- Rules and past data

In simple terms, the system uses AI, backend logic, and a graph database to analyze changes and generate results.



This diagram shows how frontend, backend, AI, and database interact.

3. Technology Stack & Requirements

3.1 Technology Stack

The system is built using the following technologies:

- **Frontend:** React.js (for user interface)
- **Backend:** FastAPI (Python-based API framework)
- **Database:** Neo4j (Graph database for system relationships)
- **AI Integration:** Google Gemini API (for intent extraction)
- **Styling:** Tailwind CSS

3.2 Software Requirements

To run the project, the following software should be installed:

- **Node.js:** v18 or above
- **Python:** v3.9 or above
- **Neo4j:** v5.x (Desktop or Server)
- **Git:** Latest version

3.3 Python Dependencies

Install using `requirements.txt`. Common libraries include:

- FastAPI
- Uvicorn
- Neo4j
- Pydantic
- Numpy

3.4 Frontend Dependencies

Install using `npm install`. Key libraries:

- React
- Axios (for API calls)
- Tailwind CSS

3.5 Environment Configuration

Create a `.env` file with the following variables:

- GEMINI_API_KEY
- NEO4J_URI
- NEO4J_USERNAME
- NEO4J_PASSWORD

3.6 System Requirements

- OS: Windows / Linux / Mac
- RAM: Minimum 8 GB recommended
- Browser: Chrome / Edge

4. New Developer Setup Guide

4.1 Clone the Project

Clone the repository and open the project folder.

4.2 Backend Setup

- Create virtual environment
- Activate it
- Install dependencies

4.3 Environment Setup

Create `.env` file and add:

- Gemini API Key
- Neo4j URL, username, password

4.4 Database Setup (Neo4j)

- Install Neo4j
- Start database
- Load graph data

This database stores connections between systems, which helps in identifying how one change affects other systems.

4.5 Run Backend

Start FastAPI server.

4.6 Frontend Setup

- Go to frontend folder
- Install packages
- Start React app

4.7 Check Everything

- Open browser
- Enter sample input
- Check output (impact, cost, rules)

5. How to Add a New Feature

5.1 Understand Requirement

Clearly understand what feature needs to be added and what output is expected.

5.2 Backend Changes

- Add or update APIs
- Update logic

5.3 AI Changes

- Update prompt if needed
- Improve output format

5.4 Database Changes

- Add new nodes or connections
- Update graph

5.5 Frontend Changes

- Update UI
- Connect APIs

5.6 Testing

- Test different inputs
- Check:
    - Impact

    - Cost
    - Rules

5.7 Documentation

Update documents and add examples.

6. System Workflow

1. User enters design change
2. AI understands input
3. Backend processes it
4. Neo4j finds affected systems
5. Cost and rules are calculated
6. Result is shown on UI

This workflow helps in automating the complete impact analysis process.

7. Execution Flow After Setup

After setup:

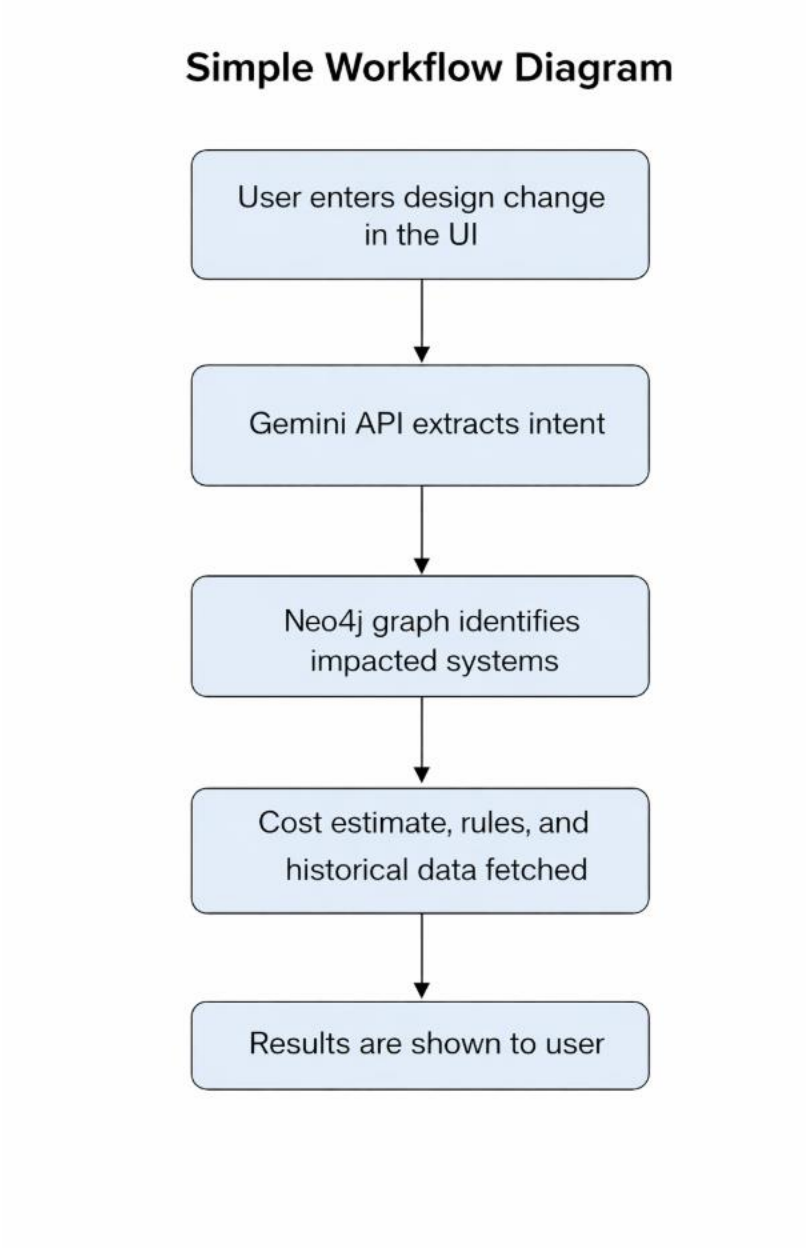
1. Start backend
2. Start frontend
3. Open app in browser
4. Enter input
5. Click **Analyze**

Then system internally:

- Sends input to AI
- Converts it into structured data
- Finds affected systems from Neo4j
- Calculates cost (P10, P50, P90)
- Checks rules and past data

These steps are executed automatically without user intervention.

This means that once the system is set up, any user can easily run and analyze design changes without manual effort.

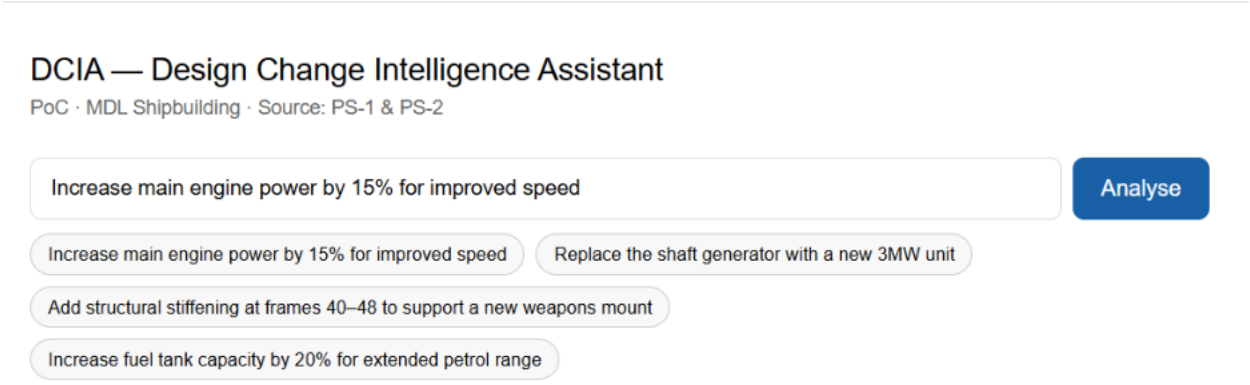


8. Sample Output Explanation

The system provides clear and structured output to help users understand the impact of the change.

Example input: **“Increase main engine power by 15% for improved speed”**

System gives:



8.1 Impact

- Total systems affected: 6
- Examples:
    - Hull → High
    - Electrical → Medium
    - Shafting → High
    - Fuel → Medium

8.2 Cost

- P10 → Low cost
- P50 → Average cost
- P90 → High cost

This helps in understanding risk.

8.3 Rules

- 4 rules triggered
- Based on:
    - Power limits
    - Structure
    - Electrical load

8.4 Past Data

- Shows similar previous cases
- Helps in better decision

8.5 Final Analysis

System tells:

- What systems are affected
- What risks are there
- Is change safe or risky

The system summarizes the overall impact, highlights risks, and helps decide whether the change is safe or not.

9. Output Summary

Final output includes:

- Affected systems
- Cost estimation
- Rules
- Past data
- AI analysis

Overall, the system converts complex design analysis into a simple, fast, and automated process.

Impact assessment

Design Change Impact Assessment:

User Request:
Increase main engine power by 15% for improved speed

Key Findings:
- Total impacted systems: 6
- Class rules triggered: 4
- Historical analogues found: 1

Cost Estimate (USD):
- P10 (Optimistic): $4,947,188
- P50 (Most Likely): $6,596,250
- P90 (Pessimistic): $8,904,938

Analysis:
The proposed change has cascading impacts across multiple ship systems.
Critical dependencies across propulsion, structure, and electrical systems
should be carefully reviewed before implementation.

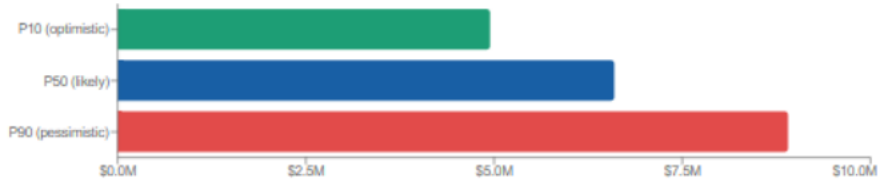
Conclusion:
The design change is feasible with moderate cost and manageable risk.
Further validation using detailed engineering analysis is recommended.

(Note: This is a mock AI-generated report for PoC demonstration.)
Source: Neo4j graph traversal + LLM synthesis · Confidence: MEDIUM

Affected systems (6)

System	Discipline	Hops from origin	Severity	Reason
Hull Structure	Structure	1	HIGH	—
Electrical Power	Electrical	1	MEDIUM	—
Shafting & Gearbox	Propulsion	1	HIGH	—
Fuel System	Outfitting	1	MEDIUM	—
Outfitting / HVAC	Outfitting	2	LOW	—
Weapons Systems	Weapons	2	LOW	—

Cost estimate (USD)



PoC parametric model — replace with MDL historical data regression in production

Class rule flags (4)

LR — Pt3 Ch2 Sec5.3
Main engine power-to-weight ratio limits for naval vessels
Triggered by: Main Propulsion

DNV — RU-SHIP Pt4 Ch2
Vibration limits for hull structure linked to propulsion
Triggered by: Hull Structure

DNV — RU-SHIP Pt4 Ch6
Electrical load balance requirements after machinery change
Triggered by: Electrical Power

LR — Pt3 Ch4 Sec2.1
Shaft alignment tolerances post power upgrade
Triggered by: Shafting & Gearbox

Historical analogues

HC-2022-014 — Engine upgrade from MTU 3600 to MTU 4000 on OPV class
Power increase 12%. Required shaft re-alignment, electrical load re-calc, hull stiffening at frames 42-46.
Actual cost: $3,200,000 · Schedule delay: 28 days

10. Conclusion

The system is currently in the Proof of Concept (PoC) stage but is working effectively. It supports easy onboarding, faster development, and scalability.

11. Future Improvements

- Use real-world data

- Improve AI accuracy
- Better UI dashboards
- Production deployment

In the future, this system can evolve into a fully production-ready intelligent decision support system.

12. Key Highlights

- AI-based input understanding
- Graph-based impact analysis
- Cost prediction (P10, P50, P90)
- Modular system
- Real-time analysis